

Programación de Sistemas

Unidad 1: Introducción a la Programación de Sistemas y los Compiladores

Nota de clase 01: Panorama de la programación de sistemas

Manuel Soto Romero

Araceli Liliana Reyes Cabello

Semestre 2026-2

Colegio de Ciencia y Tecnología UACM

En un curso inicial de programación solemos concentrarnos en escribir instrucciones que resuelvan un problema. Sin embargo, entre el texto escrito por una persona y su ejecución en una computadora intervienen otros programas: algunos traducen, otros reúnen partes separadas, otros preparan el programa en memoria y otros administran los recursos necesarios para ejecutarlo. En esta nota estudiaremos ese entorno como un panorama: definiremos la programación de sistemas, reconoceremos sus áreas principales y ubicaremos a los compiladores e intérpretes dentro de ella. Finalmente, explicaremos por qué la construcción de un compilador será el caso de estudio que articulará el curso.

Ruta de lectura (15–20 minutos)

Lectura esencial: sigue el caso inicial; localiza las definiciones y comparaciones de las secciones 1 y 3; reconoce las responsabilidades y el recorrido panorámico de la sección 2; y cierra con el esquema $\text{IMP/IMP++} \rightarrow \text{C}$ y las cuatro ideas de la sección 4.

Lectura complementaria: vuelve después a los matices sobre la frontera entre aplicaciones y software de sistemas, las combinaciones entre traducción e interpretación y las observaciones que amplían el panorama. La conclusión sirve como comprobación final del recorrido esencial.

Caso inicial. Del archivo fuente a la ejecución

Supongamos que una persona escribe este programa en C:

```
int main(void) {  
    return 0;  
}
```

El archivo contiene una descripción comprensible para quien conoce el lenguaje, pero el procesador no ejecuta directamente ese texto. Para llegar a la ejecución se necesita un conjunto de herramientas que transforme el programa y lo prepare para la máquina. El programa de aplicación expresa *qué* se quiere hacer; el software de sistemas participa en *cómo* hacerlo ejecutable.

1. Definición y características de la programación de sistemas

El software de sistemas comprende programas que apoyan la operación de una computadora. Gracias a ese apoyo, una persona puede concentrarse en el problema que desea resolver sin controlar directamente todos los detalles internos de la máquina. Un editor permite preparar el texto de un programa; un traductor cambia su representación; un cargador lo coloca en memoria; un sistema operativo coordina recursos. Cada herramienta cumple una función distinta, pero todas forman parte del entorno que permite usar la computadora.

Definición 1. (Software de sistemas)

El **software de sistemas** es el conjunto de programas que apoya la operación y el uso de una computadora, y que proporciona servicios para preparar, traducir, ejecutar o controlar otros programas.

La **programación de sistemas** se ocupa del diseño y la implementación de ese tipo de software. Su objeto no es una sola aplicación, sino los mecanismos que permiten que los programas se relacionen con lenguajes, herramientas y recursos de cómputo.

Programación de aplicaciones	Programación de sistemas
Se concentra en un problema de una persona o una organización.	Se concentra en la infraestructura de software que permite preparar y ejecutar programas.
Su producto suele utilizarse para realizar una tarea concreta.	Su producto suele proporcionar servicios a otros programas o administrar recursos de la computadora.
Pregunta típica: ¿qué resultado necesita la persona usuaria?	Pregunta típica: ¿qué mecanismos necesita el programa para poder ejecutarse?

Esta comparación no establece una frontera absoluta ni una jerarquía. Una aplicación utiliza servicios del sistema y una herramienta de sistemas también debe resolver necesidades de sus usuarios. La diferencia está en el foco principal del software.

Entre las características de la programación de sistemas destacan las siguientes:

- ★ **Apoya la operación de otros programas.** Sus componentes preparan código, proporcionan servicios o administran recursos necesarios para la ejecución.
- ★ **Relaciona diferentes niveles de abstracción.** Conecta representaciones cercanas a las personas con representaciones que una máquina puede procesar.
- ★ **Considera la arquitectura de la computadora.** Parte de su diseño depende de instrucciones, registros, memoria y otros recursos de la máquina; otra parte puede conservar principios independientes de una arquitectura particular.
- ★ **Exige interfaces claras entre herramientas.** La salida producida por un componente debe tener la forma esperada por el siguiente.
- ★ **Debe atender corrección y manejo de errores.** Una herramienta de sistemas no sólo transforma información: también necesita detectar condiciones que impiden continuar y comunicarlas de manera útil.

Observación 1. Una mirada detrás de la ejecución

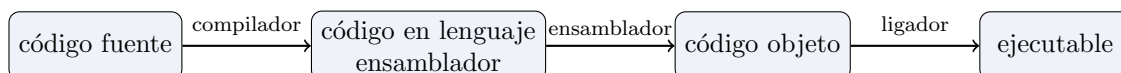
Cuando un programa se ejecuta correctamente, muchas herramientas de sistemas pasan inadvertidas. Estudiarlas significa mirar los procesos que ocurren “detrás” de la aplicación: traducción, preparación, carga, control de recursos y apoyo para localizar errores.

2. Principales áreas de la programación de sistemas

La programación de sistemas abarca varias familias de herramientas. Entre sus áreas principales se encuentran los ensambladores, cargadores y ligadores, procesadores de macros, compiladores y sistemas operativos; también incluye otras herramientas como editores de texto y depuradores interactivos. En esta nota sólo identificaremos su función general.

Área o herramienta	Función general
Ensambladores	Traducen programas escritos en lenguaje ensamblador a código objeto o código de máquina.
Ligadores y cargadores	Los ligadores reúnen código objeto y bibliotecas para formar un programa ejecutable; los cargadores colocan el programa en memoria y lo preparan para su ejecución.
Procesadores de macros	Reconocen abreviaturas definidas como macros y las expanden en secuencias del lenguaje correspondiente.
Compiladores e intérpretes	Procesan programas escritos en un lenguaje: un compilador produce un programa en otro lenguaje; un intérprete ejecuta las operaciones indicadas sin entregar, como resultado principal, un programa destino separado.
Sistemas operativos	Administran recursos de la computadora y proporcionan servicios para la ejecución de programas.
Editores y depuradores	Los editores apoyan la creación y modificación de programas; los depuradores ayudan a observar su ejecución y localizar errores.

Estas herramientas no trabajan necesariamente de forma aislada. En un sistema de procesamiento de lenguajes, la salida de una puede convertirse en la entrada de otra. Por ejemplo, un recorrido panorámico frecuente es el siguiente:



El cargador coloca el ejecutable en memoria para iniciar su ejecución.

El esquema sirve para reconocer cooperación entre herramientas, no como una receta universal. La forma concreta del recorrido depende de los lenguajes, la plataforma y las decisiones del sistema.

Observación 2. Área no significa programa aislado

Hablar de áreas ayuda a distinguir responsabilidades. En un entorno real, varias de ellas forman una cadena: cada herramienta recibe una representación, realiza una tarea y produce una nueva representación o prepara la ejecución.

3. Compiladores e intérpretes como software de sistemas

Los compiladores y los intérpretes pertenecen a una familia más amplia: los **procesadores de lenguajes**. Ambos permiten trabajar con un programa fuente, pero lo hacen de manera diferente.

Definición 2. (Compilador)

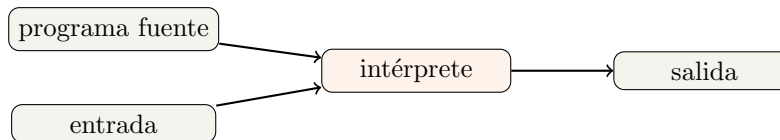
Un **compilador** es un programa que lee un programa escrito en un lenguaje fuente y lo traduce a un programa equivalente escrito en un lenguaje destino. Durante la traducción también informa los errores que puede detectar en el programa fuente.

La palabra *equivalente* indica que el programa destino debe conservar el comportamiento definido por el programa fuente. Los dos programas no tienen que verse iguales ni usar las mismas instrucciones; deben corresponder en lo que hacen.



Definición 3. (Intérprete)

Un **intérprete** es un programa que ejecuta las operaciones especificadas por un programa fuente a partir de las entradas proporcionadas, sin producir como resultado principal un programa destino independiente.



	Compilador	Intérprete
Entrada principal	Programa fuente.	Programa fuente y datos de entrada.
Acción principal	Traduce el programa.	Ejecuta las operaciones del programa.
Resultado principal	Programa destino y mensajes de error, si se detectan.	Resultados de la ejecución y mensajes de error, si se detectan.

La distinción anterior presenta los modelos básicos. En la práctica existen sistemas que combinan traducción e interpretación. Lo importante en este punto es reconocer la diferencia entre *producir otra representación ejecutable* y *ejecutar las operaciones del programa fuente*.

Ejemplo conceptual. Dos maneras de procesar un programa

Imaginemos un programa que recibe un número y muestra su doble.

- ★ Con un **compilador**, primero se obtiene un programa destino. Después, ese programa se ejecuta con distintos números de entrada.
- ★ Con un **intérprete**, el programa fuente y cada número de entrada son procesados para producir directamente el resultado de esa ejecución.

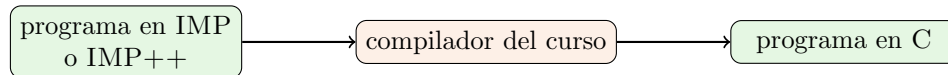
El cálculo solicitado es el mismo; cambia la manera en que el procesador de lenguajes lleva el programa a la ejecución.

4. El compilador como caso de estudio integrador del curso

Un compilador es una herramienta adecuada para estudiar programación de sistemas porque conecta varios problemas en un solo producto. Debe reconocer un lenguaje fuente, detectar errores, construir

y transformar representaciones, considerar el lenguaje destino y organizar componentes que colaboran mediante interfaces precisas. Su estudio relaciona lenguajes de programación, arquitectura de computadoras, teoría de lenguajes, algoritmos e ingeniería de software.

En las notas y clases construiremos progresivamente un compilador para el IMP de Winskel. Las prácticas partirán de esa versión funcional y la extenderán para construir IMP++. Ambos recorridos tendrán como lenguaje destino a C. Por ahora basta representar el objetivo general como una transformación:



IMP++ no será un compilador separado: conservará las construcciones de IMP y añadirá, de manera acumulativa, las extensiones aprobadas para las prácticas.

La construcción acumulativa permitirá observar cuatro ideas propias de la programación de sistemas:

1. **Especificar.** Una herramienta no puede procesar correctamente un lenguaje si sus reglas no están definidas con claridad.
2. **Transformar.** El programa cambia de representación sin perder el comportamiento que debe conservarse.
3. **Dividir responsabilidades.** Un sistema complejo se organiza en componentes con entradas y salidas explícitas.
4. **Comprobar.** Es necesario probar programas correctos, detectar errores y verificar el resultado producido.

Observación 3. El compilador es el hilo conductor

El curso no pretende revisar todas las áreas de la programación de sistemas con la misma profundidad. La implementación guiada de IMP y su extensión a IMP++ formarán un solo producto acumulativo para estudiar, de manera concreta, cómo se especifica, divide, implementa y prueba una herramienta de sistemas.

Conclusión

La programación de sistemas se ocupa de herramientas que apoyan la operación de la computadora y hacen posible preparar, traducir, ejecutar y controlar programas. Entre sus áreas se encuentran ensambladores, ligadores, cargadores, procesadores de macros, compiladores, intérpretes, sistemas operativos, editores y depuradores. Dentro de este panorama, el compilador destaca porque transforma un programa fuente en un programa destino y reúne problemas de especificación, traducción, modularidad, errores y comprobación. En el siguiente tema abriremos esta herramienta para reconocer su estructura general y la relación entre sus componentes.

Referencias

- [1] Beck, L. L. (1996). *System Software: An Introduction to Systems Programming* (3.^a ed.). Addison-Wesley. ISBN: 978-0-201-42300-6.

- [2] Bala, K., Bracy, A., Sirer, E. y Weatherspoon, H. (2025). *Compilation—Assemblers, Linkers, & Loaders*. CS 3410: Computer System Organization and Programming [diapositivas]. Archivo `14-compilation-notes.pdf`.
- [3] Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2.^a ed.). Pearson/Addison-Wesley.
- [4] Vilar Torres, J. M. (2010). *Estructura de los compiladores e intérpretes*. Universitat Jaume I.